Computational Logic

Propositional Logic

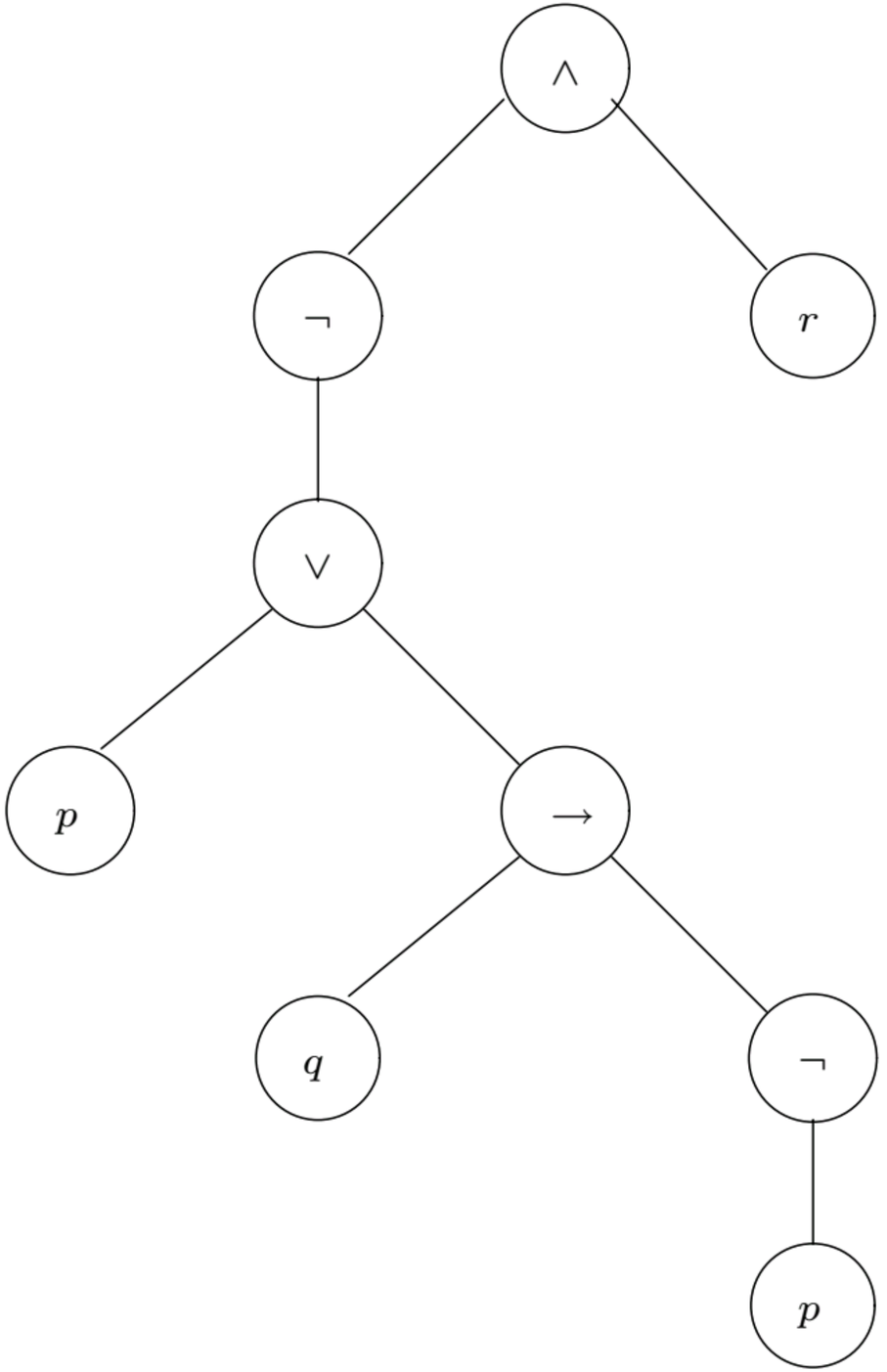
Jakub Szymanik

- Week 1 (14.09): Intro (try to read Logicomix after)
- Week 2 (21.09): How to Win a Million Dollars with Logic? Part 1:The language and what it means. (Study Sections 1.1, 1.3, 1.4.1-1.4.2 of the book)
- Week 3: (28.09) How to win a million dollars with logic? Part 2: Proof - and why it matches meaning. (Study 1.2, 1.4.3-1.4.4)
- Week 4: (05.10) How to win a million dollars with logic? Part 3: Normal forms and solvers. Lab, BY laptops (Study 1.5, 1.6)
- Homework 1.
- Week 5: (12.10) FO and non-computability (Study 2.1-2.2)
- Week 6: (19.10) FO and non-computability (Study 2.3-2.4)
- Week 7: (26.10) FO and non-computability (Study 2.5) + Trial intermediate exam.
- Homework 2.
- Week 8: (02.11) Expressiveness, Definability, and Generalized Quantifier Theory (Study 2.6-27, and the book chapter on Moodle)
- Week 9: (09.11) Modal Logic and Agency 1 (Study 5.2-5.3)
- Week 10: (16.11) Modal Logic and Agency 2 (Study 5.4-5.5)
- Week 11: (23.11) Modal Logic and Agency 3.
- Homework 3.
- Week 12: (30.11) Q&A + Trial final exam.

Syntax

PL as a formal language

- Set of basic facts called atomic sentences: AT={p, q, r, s, t,…}
- The set of well-formed formulas (wff) is the smallest set containing:
- All atomic sentences
- If x and y are in wff then also: ¬x, (x ∧ y), (x ∨ y), (x→y), and (x↔y) are in wff


∧
¬
r
∨
p
→
q
¬
p

E1

Which are well-formed, given the definition on the slide?

(a) $\neg(p \wedge q)$

(b) $(p \wedge q \vee r)$

(c) $((p \rightarrow q) \rightarrow (q \rightarrow p))$

(d) $\neg\neg\neg p$ (e) $(p \neg q)$

Then draw the parse tree of $\neg(p \wedge (q \rightarrow \neg r))$.

A1

(a) ✓

(b) ✗ — no way to build it: $\wedge$ and $\vee$ would need brackets to say which is the main connective

(c) ✓

(d) ✓

(e) ✗.

Moral: the grammar is a program, and (b) fails for exactly the reason a parser would reject it — ambiguity, not ugliness. Compare with the solutions manual, Exercises 1.3, 1(d) and 1(f), whose trees are worked out.

Intuitive interpretation

Meaning

- ¬x - negation
- (x ∧ y) - conjunction (and)
- (x ∨ y) -disjunction (or)
- (x→y) - conditional/implication (if…then)
- (x↔y) - biconditional (if and only if)

Compositional Semantics

- Assignment, v: AT$\longrightarrow${0, 1} (false, true)
- **Principle of compositionality:** the meaning of a complex expression is determined by the meanings of its constituent expressions and the rules used to combine them.

Are NNs capable of compositional interpretation?

Compositionality decomposed: how do neural networks generalise?

Dieuwke Hupkes, Verna Dankers, Mathijs Mul, Elia Bruni

Despite a multitude of empirical studies, little consensus exists on whether neural networks are able to generalise compositionally, a controversy that, in part, stems from a lack of agreement about what it means for a neural model to be compositional. As a response to this controversy, we present a set of tests that provide a bridge between, on the one hand, the vast amount of linguistic and philosophical theory about compositionality of language and, on the other, the successful neural models of language. We collect different interpretations of compositionality and translate them into five theoretically grounded tests for models that are formulated on a task-independent level. In particular, we provide tests to investigate (i) if models systematically recombine known parts and rules (ii) if models can extend their predictions beyond the length they have seen in the training data (iii) if models' composition operations are local or global (iv) if models' predictions are robust to synonym substitutions and (v) if models favour rules or exceptions during training. To demonstrate the usefulness of this evaluation paradigm, we instantiate these five tests on a highly compositional data set which we dub PCFG SET and apply the resulting tests to three popular sequence-to-sequence models: a recurrent, a convolution-based and a transformer model. We provide an in-depth analysis of the results, which uncover the strengths and weaknesses of these three architectures and point to potential areas of improvement.

- *recursiveness* (elements can be iteratively combined)
- *compositionality* (structures are determined by their constituents)

See: https://aclanthology.org/2024.findings-naacl.147.pdf

Truth-functional semantics for complex formulas

- $v(\neg x)=1$ iff $v(x)=0$
- $v(x \wedge y)=1$ iff $v(x)=1$ and $v(y)=1$
- $v(x \vee y)=1$ iff $v(x)=1$ or $v(y)=1$
- $v(x \rightarrow y)=1$ iff $v(x)=0$ or $v(y)=1$
- $v(x \leftrightarrow y)=1$ iff $v(x)=v(y)$

Truth-tables

x	y	¬x	x ∧ y	x ∨ y	x → y	x ↔ y
1	1	0	1	1	1	1
1	0	0	0	1	**0**	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

→ *is false in exactly one row.*

The conditional

- | p | q | p → q | → 1 1 1 · 1 0 0 · 0 1 1 · 0 0 1
- Only one row makes it false: true antecedent, false consequent
- "If 2+2=5, then I am the Pope." — true
- "Every student in this room over 3 m tall is a millionaire." — true
- → is not causation, relevance, or time. It is one row of a table
- Remember the cards: people turn the wrong ones because they read → as ↔

E2

$v(p)=1$, $v(q)=0$, $v(r)=1$.

Evaluate $((p \rightarrow q) \vee (\neg r \leftrightarrow q))$ using the parse tree, bottom-up.

A2

$p \rightarrow q = 0.\ \neg r = 0.\ 0 \leftrightarrow 0 = 1.\ 0 \vee 1 = 1$

The reason to insist on the tree: this is compositionality doing visible work — each node's value depends only on its children's.

E3

(i) Find a valuation that makes $p \rightarrow q$ true and $q \rightarrow p$ false.

(ii) Is $(p \rightarrow q) \vee (q \rightarrow p)$ ever false?

A3

(i) p=0, q=1.

(ii) Never — it's a tautology, which most people find deeply weird: for any two unrelated sentences, one of them implies the other. It’s a clean demonstration that → is not "relevance".

E4

(i) How many truth functions of n arguments?

(ii) Write XOR using only $\wedge$, $\vee$, $\neg$.

(iii) Now using only NAND.

A4

(i) 2^(2ⁿ) — 16 for n=2, 256 for n=3.

THE 16 BINARY CONNECTIVES		X Y			
		11	**10**	**01**	**00**
⊤	tautology	1	1	1	1
x ∨ y	disjunction	1	1	1	0
y → x	converse implication	1	1	0	1
x	projection	1	1	0	0
x → y	implication	1	0	1	1
y	projection	1	0	1	0
x ↔ y	biconditional	1	0	0	1
x ∧ y	conjunction	1	0	0	0
x ↑ y	NAND	0	1	1	1
x ⊕ y	XOR	0	1	1	0
¬y	negation of y	0	1	0	1
x ∧ ¬y	non-implication	0	1	0	0
¬x	negation of x	0	0	1	1
¬x ∧ y	converse non-implication	0	0	1	0
x ↓ y	NOR	0	0	0	1
⊥	contradiction	0	0	0	0

the ones we use each one on its own is enough for all the others the prisoners' parity function

A4 c’td

(ii) $(p \vee q) \wedge \neg(p \wedge q)$, or $(p \wedge \neg q) \vee (\neg p \wedge q)$.

(iii) $x \oplus y = (x \uparrow (x \uparrow y)) \uparrow (y \uparrow (x \uparrow y))$

$A = x \uparrow y$

$B = x \uparrow A$

$C = y \uparrow A$

$x \oplus y = B \uparrow C$

$B = \neg(x \wedge \neg(x \wedge y)) = \neg(x \wedge \neg y)$ — "it is not the case that x alone holds"

$C = \neg(y \wedge \neg(x \wedge y)) = \neg(\neg x \wedge y)$ — "it is not the case that y alone holds"

$B \uparrow C = \neg(B \wedge C) = (x \wedge \neg y) \vee (\neg x \wedge y)$ — "one of them holds alone"

Tautologies, contradictions, contingencies, and satisfiability

- A formula that is always true is called a tautology.
- A formula that is always false is called a contradiction.
- All other formulas are sometimes called contingencies.
- A formula is satisfiable if it has a valuation in which it evaluates to 1.

E5

Classify each, and justify:

(a) $p \lor \neg p$

(b) $p \land \neg p$

(c) $(p \to q) \to (\neg q \to \neg p)$

(d) $(p \to q) \to (q \to p)$

(e) $((p \to q) \land p) \to q$

A5

a) tautology; $(p \vee \neg p)$

(b) contradiction; $(p \wedge \neg p)$

(c) tautology — contraposition; $(p \to q) \to (\neg q \to \neg p)$

(d) contingent: p=0, q=1 gives $1 \to 0 = 0$; $(p \to q) \to (q \to p)$

(e) tautology — modus ponens; $((p \to q) \wedge p) \to q$

Million Dollar Question

- How many rows are in the table for the formula containing *n* atoms?
- No, that’s not a million-dollar question.
- Is there a faster method to check whether a formula is satisfiable?
- This is a US$1,000,000 question and one of the seven biggest open questions in CS.



https://www.claymath.org/millennium/p-vs-np/

Substitutions

- In tautologies and contradictions, any statement may be uniformly substituted for the atomic statements without altering the truth value of the initial formula.
- So, tautologies (contradictions) are true (false) by their form.

Logical equivalence

- P and Q are logically equivalent iff $P \leftrightarrow Q$ is a tautology.
- Substitution of logically equivalent expressions preserves the truth-value.

Logical consequence

- Q is a logical consequence of P iff $P \rightarrow Q$ is a tautology.
- The basis for constructing valid arguments.
- We can only substitute Q for P with the guarantee of preserving the truth.

Valid arguments

- An argument consists of premises, P1, P2,…, Pn, and a conclusion Q.
- An argument is valid iff ((P1 $\wedge$ P2 $\wedge$… $\wedge$ Pn)$\rightarrow$Q) is a tautology ($\models$)
- A proof of Q from P_1,…,P_n is a finite sequence of lines, each of which is either a premise, or a temporary assumption, or follows from earlier lines by a rule — and which ends with Q with no assumption left open. (What "no assumption left open" means is next week's subject.)

E6

Formalize and test:

- (A) If the server is down, the page won't load. The page won't load. So the server is down.
- (B) If the server is down, the page won't load. The page loads. So the server is not down.

A6

With p = "server down", q = "page won't load”:

(A) is $p \rightarrow q,\ q \models p$ — invalid, p=0 q=1 (the page can fail for other reasons). That's affirming the consequent, and it is the single most common fallacy in incident triage.

(B) is $p \rightarrow q,\ \neg q \models \neg p$ — valid, modus tollens

Two turnstiles

$P_1,\ldots,P_n \models Q$ — every valuation making all premises true makes Q true (semantics: check every row)

$P_1,\ldots,P_n \vdash Q$ — there is a derivation of Q from the premises (syntax: a finite object, no meanings involved)

Two completely different definitions. No reason they should coincide.

Soundness: $\vdash$ implies $\models$ — the rules never lie

Completeness: $\models$ implies $\vdash$ — the rules miss nothing

Both proved in week 3 (§1.4.3–1.4.4). Until then, treat them as unrelated.

Natural Deduction

The basic rules of natural deduction:

	introduction	*elimination*
$\wedge$	$\dfrac{\phi \quad \psi}{\phi \wedge \psi}\ \wedge\mathrm{i}$	$\dfrac{\phi \wedge \psi}{\phi}\ \wedge\mathrm{e}_1 \qquad \dfrac{\phi \wedge \psi}{\psi}\ \wedge\mathrm{e}_2$
$\vee$	$\dfrac{\phi}{\phi \vee \psi}\ \vee\mathrm{i}_1 \qquad \dfrac{\psi}{\phi \vee \psi}\ \vee\mathrm{i}_2$	$\dfrac{\phi \vee \psi \quad \begin{array}{c}[\phi]\\ \vdots\\ \chi\end{array} \quad \begin{array}{c}[\psi]\\ \vdots\\ \chi\end{array}}{\chi}\ \vee\mathrm{e}$
$\rightarrow$	$\dfrac{\begin{array}{c}[\phi]\\ \vdots\\ \psi\end{array}}{\phi \rightarrow \psi}\ \rightarrow\mathrm{i}$	$\dfrac{\phi \quad \phi \rightarrow \psi}{\psi}\ \rightarrow\mathrm{e}$
$\neg$	$\dfrac{\begin{array}{c}[\phi]\\ \vdots\\ \bot\end{array}}{\neg\phi}\ \neg\mathrm{i}$	$\dfrac{\phi \quad \neg\phi}{\bot}\ \neg\mathrm{e}$
$\bot$	(no introduction rule for $\bot$)	$\dfrac{\bot}{\phi}\ \bot\mathrm{e}$
$\neg\neg$		$\dfrac{\neg\neg\phi}{\phi}\ \neg\neg\mathrm{e}$

Some useful derived rules:

$$\frac{\phi \rightarrow \psi \quad \neg\psi}{\neg\phi}\ \mathrm{MT} \qquad \frac{\phi}{\neg\neg\phi}\ \neg\neg\mathrm{i}$$

$$\frac{\begin{array}{c}[\neg\phi]\\ \vdots\\ \bot\end{array}}{\phi}\ \mathrm{PBC} \qquad \frac{}{\phi \vee \neg\phi}\ \mathrm{LEM}$$

Figure 1.2. Natural deduction rules for propositional logic.

We prove the validity of $(p \wedge q) \wedge r,\ \ s \wedge t \vdash q \wedge s$ by

1	$(p \wedge q) \wedge r$	premise
2	$s \wedge t$	premise
3	$p \wedge q$	$\wedge \mathrm{e}_1$ 1
4	q	$\wedge \mathrm{e}_2$ 3
5	s	$\wedge \mathrm{e}_1$ 2
6	$q \wedge s$	$\wedge \mathrm{i}$ 4, 5

Solve exercises 1.1–1.3 (p. 78)